EXERCISE 10 – IMPLEMENTATION OF BLOCKS WORLD PROGRAM

AIM

To implement the Blocks World program and demonstrate moving blocks between stacks.

PROCEDURE

1. Define the BlocksWorld class to represent blocks and their stacks.
2. Initialize the blocks so that each block starts on the table.
3. Create functions to display the state, find a block, move blocks, and set the goal state.
4. Create the main function and specify the initial and goal states.
5. Execute the program and observe the changes in the blocks configuration.

PROGRAM

```
# Class representing the state of the blocks world
class BlocksWorld:
    def __init__(self, num_blocks):
        # Initialize blocks world with num_blocks on the table
        self.state = [[block] for block in range(num_blocks)]
        self.num_blocks = num_blocks

    def display_state(self):
        # Display the current state of the blocks world
        for stack in self.state:
            print(f"Block(s) on stack: {stack}")

    def move(self, block, destination):
        # Move block onto another block or the table
        source_stack = self.find_block(block)
        destination_stack = self.find_block(destination)
        if source_stack is None or destination_stack is None:
            print(f"Invalid block {block} or destination {destination}.")
            return
        # Remove the block from its current position
        source_stack.remove(block)
        # Put the block on the destination stack
        destination_stack.append(block)
        # Display the state after the move
        self.display_state()

    def find_block(self, block):
        # Find the stack that contains the given block
        for stack in self.state:
            if block in stack:
                return stack
        return None

    def goal_state(self, goal):
        # Goal state is a list of stacks representing the desired configura
        self.state = goal
        print("Goal state set.")
```

```
        self.display_state()

# Main program to solve Blocks World problem
def main():
    # Initialize Blocks World with 3 blocks (0, 1, 2)
    blocks_world = BlocksWorld(3)
    print("Initial state:")
    blocks_world.display_state()

    # Define the goal state
    goal = [[0, 1], [2]]
    blocks_world.goal_state(goal)

    print("\nPerforming Moves:")
    # Move Block 0 onto Block 2
    blocks_world.move(0, 2)
    # Move Block 1 onto Block 2
    blocks_world.move(1, 2)
    # Move Block 2 onto Block 0
    blocks_world.move(2, 0)

if __name__ == "__main__":
    main()
```

OUTPUT

```
Initial state:
Block(s) on stack: [0]
Block(s) on stack: [1]
Block(s) on stack: [2]
Goal state set.
Block(s) on stack: [0, 1]
Block(s) on stack: [2]
Performing Moves:
Block(s) on stack: [1]
Block(s) on stack: [2, 0]
Block(s) on stack: []
Block(s) on stack: [1]
Block(s) on stack: [2, 0]
Block(s) on stack: []
Block(s) on stack: [1, 2]
Block(s) on stack: [0]
Block(s) on stack: []
```